Application Development

Lecture 6: Designing Object-Oriented Applications

Konstantin Kapinchev
University of Greenwich

Application Development
Lecture 6: Designing Object-Oriented Applications

In this lecture:

- Overview
- Software Requirements Specifications
- Unified Modeling Language
- Class Diagrams
- Use Case Diagrams
- Case Study
- Conclusions

Konstantin Kapinchev

Application Development
Lecture 6: Designing Object-Oriented Applications

Overview

- The design process is a key stage of the application development
- It can determine the success of the entire development process
- The actual design of a software product can be represented with a UML diagram
- In most cases, the step before producing the UML diagram is writing a software requirements specification

Konstantin Kapinchev

Software Requirements Specification

- This is a document, which describes in detail a software product[1] before the start of its design and development
- The document is written in a formal language
- Usually, the document follows a generally adopted structure[2]
- It lays the groundwork for the software development process

[1]Can apply to any software product, from a simple mobile app to a large-scale complex system
[2]Generally, the stakeholders can decide whether to follow or not a specific structure

Software Requirements Specification *continued*

- SRS helps estimate the time and cost of the project development
- It is usually written and agreed upon before signing a contract
- In many cases, it is part of the contract, or the appendix of the contract
- In some cases, it can be considered legally binding and used to settle disputes

Software Requirements Specification *continued*

- Guidance for writing a successful SRS
  - Do not use different terms for the same concept
  - Avoid switching between active voice and passive voice
  - Identify the correct level of detail and apply it consistently
  - Avoid ambiguity

Software Requirements Specification *continued*

- Guidance for writing a successful SRS
  - Ambiguity: text, which can be interpreted in more than one way
  - Most common types of ambiguity are lexical (when a word has two or more possible meanings), syntactic (when phrases can be translated in more than one ways), semantic (complete sentences, which can be translated in more than one way) and pragmatic (when the given context does not provide the needed information or clarification)[1]

[1]Khin Hayman, Azlin Nordin, Amelia Ismail, Suriani Sulaiman, "An Analysis of Ambiguity Detection Techniques for Software Requirements Specification"

Software Requirements Specification *continued*

- The SRS covers the following main areas:
  - Functional Requirements
    - Computation
    - Data processing
    - Services
  - Non-Functional Requirements
    - Security
    - Privacy
    - Performance
  - User Interface, System Interface, and others ...

Some of these requirements need a metric in order to be able to evaluate a software product. For example, performance in term of latency measured in milliseconds.

Software Requirements Specification *continued*

- **Functional Requirements**
  - **This set of requirements describe what the software product is expected to do and how to do it**
  - **These requirements are written from the users' point of view**
  - **The word "shall" is most commonly used when these requirements are stated:**

    "The software product shall <requirement>"

Software Requirements Specification *continued*

- **Functional Requirements**
  - **Example:**
    - **Upon starting, the system shall allow the users to select their role (manager, associate, intern) before entering their user name and password**
    - **Once the user logged in, the system shall provide the following options:**

      add a new record, view all records,
      edit existing records, delete records

Software Requirements Specification *continued*

- **Non-Functional Requirements**
  - **These requirements describe "how" the software product is expected to operate**
  - **As opposed to "what" the software product is expected to do, which is the functional requirements**

Software Requirements Specification *continued*

- **User Interface Requirements**
  - **These requirements define the way the users, or the customers, interact with the software product**
  - **Those include:**
    - **Ability to access functionality**
    - **Ability to navigate easily**
    - **Use of consistent graphical elements**
    - **Personalisation**
    - **And others …**

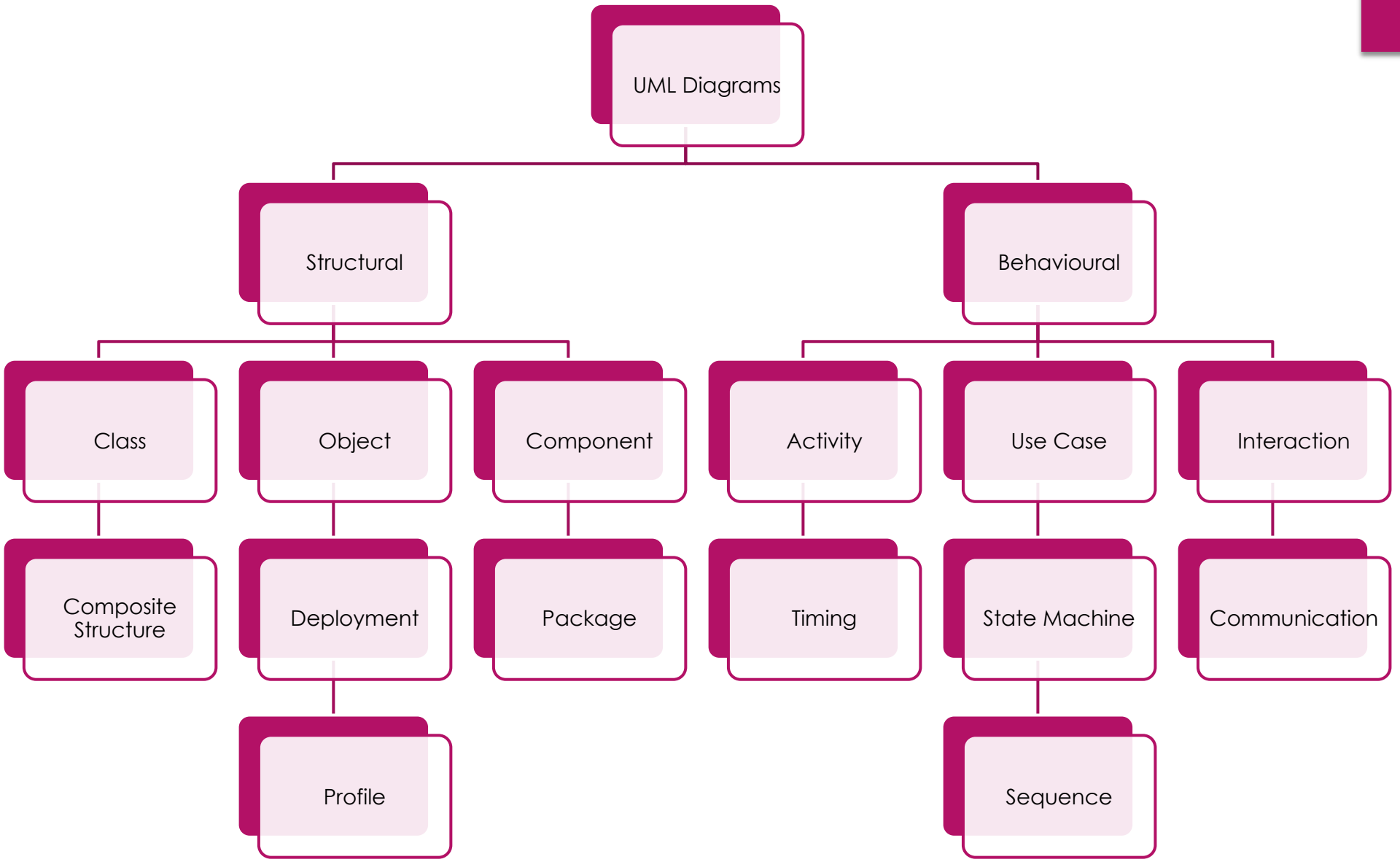
Application Development
Lecture 6: Designing Object-Oriented Applications

Unified Modeling Language

- The purpose of UML is to provide standardised notation for describing various types of systems
- Various diagramming applications which can produce UML diagrams are available
- Most software developers have some understanding of UML
- Some of the most common applications of UML include:
  - Software systems
  - Data bases
  - General workflow or other processes
  - And others ...

Konstantin Kapinchev

Types of UML Diagrams



Unified Modeling Language *continued*

- This lecture will focus on the following two types of UML diagrams:
  - UML Class Diagrams
  - UML Use Case Diagrams

Application Development
Lecture 6: Designing Object-Oriented Applications

UML Class Diagrams

- These diagrams can be used to visualise the structure of the classes of an object-oriented application
- It visualises the data and the functionality of classes
- Class diagrams show the access modifiers (visibility)
- The diagrams offer hierarchal representation based on inheritance

Konstantin Kapinchev

Application Development
Lecture 6: Designing Object-Oriented Applications

UML Class Diagrams *continued*

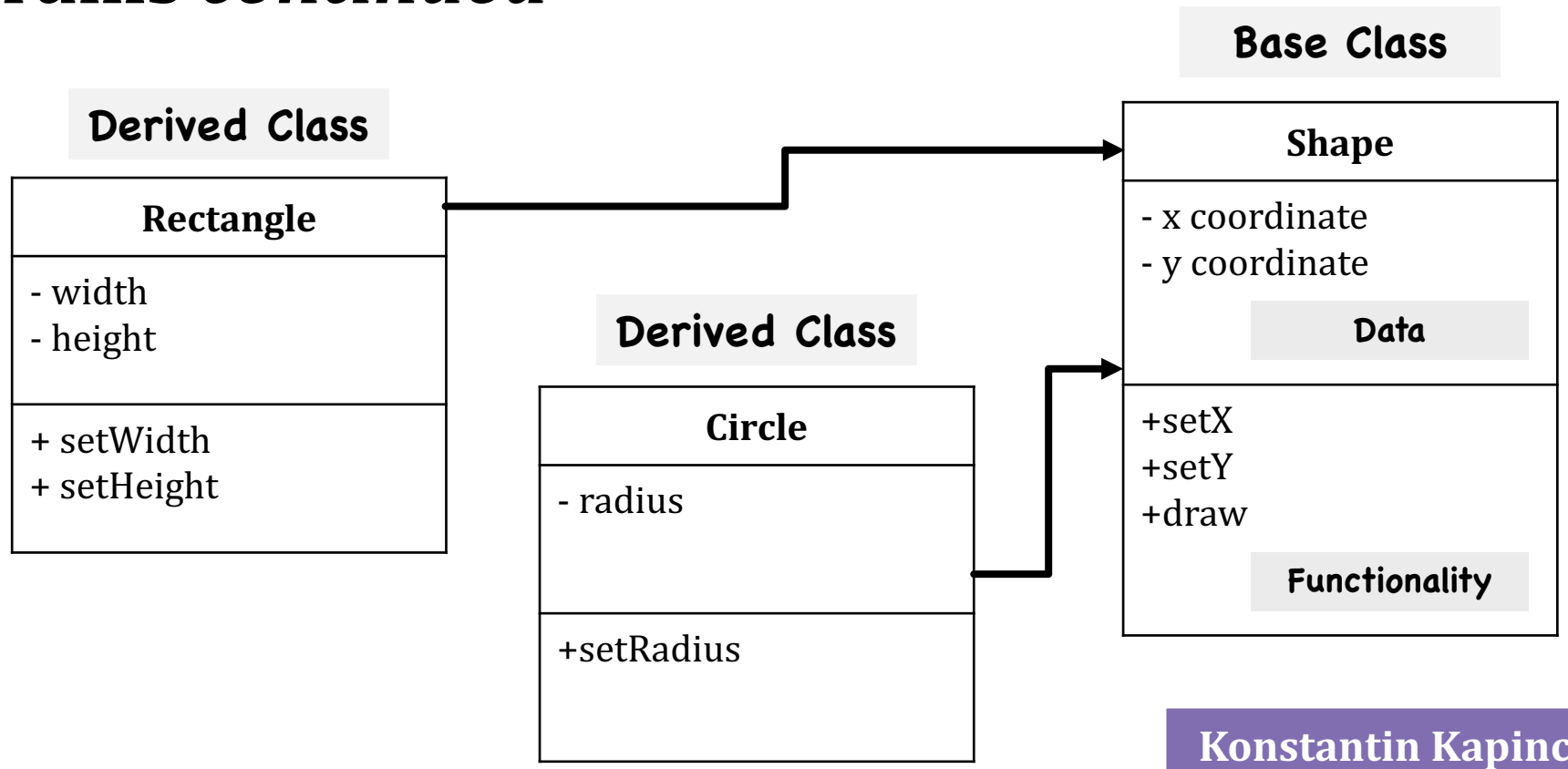
- **Elements in Class Diagrams**
  - **Classes**
  - **Data (attributes, properties, fields, variables)**
  - **Functionality (methods)**
  - **Relationships (inheritance)**

Konstantin Kapinchev

Application Development
Lecture 6: Designing Object-Oriented Applications

UML Class Diagrams *continued*

- **Example**



Visibility
- private + public # protected

Application Development
Lecture 6: Designing Object-Oriented Applications

UML Use Case Diagrams

- Use Case diagram represents graphically the interaction of a user with a system
- These diagrams provide an information about the functionality of the software product
- In most cases, these are high-level diagrams which do not provide much details, for example how exactly a specific functionality will be implemented

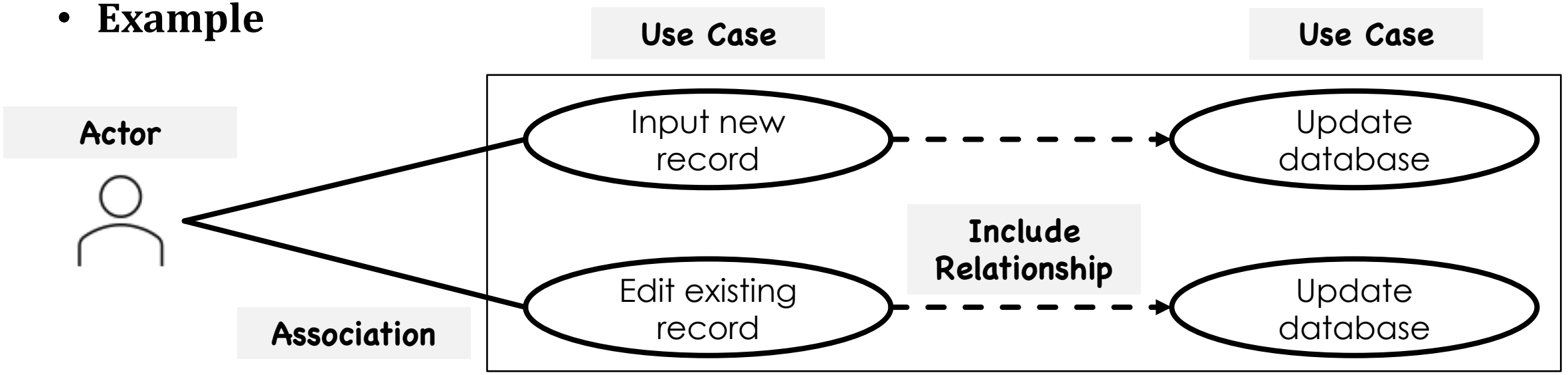
Konstantin Kapinchev

UML Use Case Diagrams *continued*

- **Elements**
  - **Actor: this is the user or the customer of the software product**
  - **Use Case: specific interaction between the user and the software product**
  - **Association: a connection between the user and a use case**
  - **Include Relationship: connects two use cases**

UML Use Case Diagrams *continued*

- **Example**



Application Development
Lecture 6: Designing Object-Oriented Applications

Case Study

- **Read the case study from:**
  Wiegars, Beatty, "Software Requirements", page 203

- **Consider the following questions:**
  - **What is the main issue?**
  - **What caused this issue?**
  - **How can this be prevented?**
  - **What would be the most significant consequence?**

Case Study *continued*

- **Read the case study from:**
  Wiegars, Beatty, "Software Requirements", page 203

- **Consider the following questions:**
  - **What is the main issue? (the software is not working as expected)**
  - **What caused this issue? (the expectations were not communicated)**
  - **How can this be prevented? (detailed SRS)**
  - **What would be the most significant consequence? (time and efforts are spent on developing a software which no one uses)**

Case Study *continued*

- Characteristics of software requirement specifications which can address the aforementioned issues[1]:
  - Completeness: providing sufficient information, covering all requirements
  - Correctness: requirements accurately describe the expected capabilities
  - Feasibility: it must be possible to implement all requirements, limitations of the hardware and software platforms are taken into account
  - Necessity: requirements should describe capabilities which are requested and expected

[1]Wiegars, Beatty, "Software Requirements", Chapter 11 Writing excellent requirements

Case Study *continued*

- **Characteristics of software requirement specifications which can address the aforementioned issues:**
  - **Priority: if possible, assign priority to each requirement according to the business needs**
  - **Unambiguity: avoid ambiguity by using technical terminology and by review the requirements document multiple times**
  - **Verification: confirm that each requirement can be tested and its functionality can be determined**

Conclusions

- This lecture discussed main steps of the software development process, namely:
  - writing software requirements specifications
  - generating UML diagrams

- In most cases, these steps are key for the success of a project

- They are produced by dedicated teams, if larger enterprises are involved

Conclusions

- **Further reading:**
  - **Wiegars, Beatty, "Software Requirements", published by Microsoft**
  - **Patrick Grassle, "UML 2.0 in Action", published by Packt**